==========
======

==Query Planner==

[PostgresPro] Query in PostgreSQL series: https://postgrespro.com/blog/pgsql/5969262

EXPLAIN command: https://www.postgresql.org/docs/current/sql-explain.html

```
EXPLAIN [ ( option [, ...] ) ] statement

where option can be one of:

    ANALYZE [ boolean ]                         -- FALSE
    VERBOSE [ boolean ]                         -- FALSE
    COSTS [ boolean ]                           -- TRUE
    SETTINGS [ boolean ]                        -- FALSE
    GENERIC_PLAN [ boolean ]                    -- FALSE
For statements with parameters
    BUFFERS [ boolean ]                         -- FALSE
Include disk usage
    SERIALIZE [ { NONE | TEXT | BINARY } ]
    WAL [ boolean ]                             -- FALSE
    TIMING [ boolean ]                          -- TRUE
    SUMMARY [ boolean ]                         -- TRUE
if ANALYZE
    MEMORY [ boolean ]                          -- FALSE
    FORMAT { TEXT | XML | JSON | YAML }         -- TEXT
```

Query plan
Plan nodes
14.2 Using EXPLAIN: https://www.postgresql.org/docs/current/using-explain.html
50.5 Planner: https://www.postgresql.org/docs/current/planner-optimizer.html
Query plan is a **tree** of **plan nodes**.
A query plan is executed by **traversing** the plan nodes.

Query planner
Cost estimate
Statistics
14.2 Statistics used by planner: https://www.postgresql.org/docs/current/planner-stats.html
68. How the planner uses statistics: https://www.postgresql.org/docs/current/planner-stats-details.html
[PostgresPro] Query execution in stages: https://postgrespro.com/blog/pgsql/5969262
Multiple query plans are possible for the same query.
Query planner seeks for the **optimal query plan**.
Query planner computes the **(estimated) cost** of each query plan to seek for the optimal.

Just In Time compilation (JIT)
30 JIT: https://www.postgresql.org/docs/current/jit.html

*** Genetic Query Optimizer**
60.1 Genetic query optimizer: https://www.postgresql.org/docs/current/geqo.html

Genetic Query Optimizer (geqo)
An **optimization algorithm** for query planning **JOIN**s.
Each JOIN query has multiple choices:

plan	Nested Loop, Hash JOIN, Merge JOIN
index	which indexes (including **multicolumn**) to use, if any

Thus, the number of possible plans grows **exponentially**:

# of plans = O(c^n)	n = # of joins

Choosing an optimal plan is **searching** every possible plan.
i.e. traveling **salesman problem**.
The theoretical **exhaustive search** is expensive.
GEQO provides an **optimized approximation**.

Genetic because it is based on a genetic problem:
given a **population** (= plan per JOIN), find the optimal **combination** that produces the **strongest gene** (=query plan)

Query planner & GEQO
By default the query planner uses theoretical exhaustive search.
It switches to GEQO only when **geqo_threshold** (default 12) is exceeded.

*** Cost estimation**
Query plan config: https://www.postgresql.org/docs/current/runtime-config-query.html

The cost consists of:
- **(table) cardinality**: table **size** i.e. total number of rows in a table
  Obtained from table **statistics**.

- **selectivity (0 ~ 1)**: the ratio of rows selected from a table
  0 = 0 rows selected
  1 = all rows selected

cost	**default**	
# disk IO		
seq_page_cost	1.0	cost of **sequential** page read from disk e.g. Seq Scan
random_page_cost	4.0	cost of **random** page

read from **disk**

		e.g. Index Scan NOTE: assumes HD

disk since random is very high

# CPU		
cpu_tuple_cost	0.01	cpu cost of **processing** a row
cpu_operator_cost	0.005	cpu cost of processing an operator (e.g. +) or function (e.g. SUM)
cpu_index_tuple_cost	0.0025	cpu cost of using an **index** in index scan
parallel_setup_cost	1000	cost of starting a parallel process
parallel_tuple_cost	0.1	cost of **moving** a row between two processes
min_parallel_table_scan_size	8MB	min data size for a **parallel scan** to be considered
min_parallel_index_scan_size	512kb	min index data size for a **parallel index scan** to be considered
effective_cache_size	4GB	
jit_above_cost	10000	
jit_inline_above_cost	50000	
jit_optimize_above_cost	50000	

*** Example: cost estimation**

[The internals of PostgreSQL] 3.2 Cost estimation: https://www.interdb.jp/pg/pgsql03/02.html#322-index-scan

```
# CREATE TABLE mytable (x int);

# INSERT INTO mytable VALUES
  (1), (2), (1), (2), (1), (3);

# CREATE INDEX ON mytable (x);

# ANALYZE;

# SELECT reltuples, relpages FROM pg_class WHERE relname
= 'mytable';
 reltuples | relpages
-----------+----------
         6 |        1

# SELECT most_common_vals, most_common_freqs FROM
pg_stats WHERE tablename = 'mytable';
 most_common_vals | most_common_freqs
------------------+-------------------
 {1,2}            | {0.5,0.33333334}
```

Seq Scan cost

```
  cost      = tuple CPU cost                                              +
page read IO cost
            = reltuples * (cpu_tuple_cost + cpu_operator_cost)        +
relpages * (seq_scan_cost)
            = 6 * (0.01 + 0.0025) + 1 * 1.0
            = 1.075
```

```
# EXPLAIN SELECT * from mytable WHERE x = 1;
                        QUERY PLAN
-----------------------------------------------------------
 Seq Scan on mytable  (cost=0.00..1.07 rows=3 width=4)
   Filter: (x = 1)
```

Index only scan cost

```
 startup    = (ceil (log reltuple) + (H + 1) * 50) * cpu_operator_cost
            = (ceil (log 6) + 50)) * 0.0025
            = 0.13

 run cost   = tuple CPU cost
            = selectivity * reltuple * (cpu_operator_cost +
cpu_index_tuple_cost)              +      ceil(selectivity * relpages) *
random_scan_cost
            = 0.5 * 6 * 0.0075  +  1 * 4.0
            = 4.0225

 total      = startup + run
            = 4.1525               (TODO: why not 4.19?)
```

```
# SET enable_seqscan to false;

# EXPLAIN SELECT * from mytable WHERE x = 1;
                                    QUERY PLAN
-----------------------------------------------------------------------
---------------------------
 Index Only Scan using mytable_x_idx on mytable
(cost=0.13..4.19 rows=3 width=4)
   Index Cond: (x = 1)
```

==========
======

== EXPLAIN operations ==

Note: at the time of writing (2025/03) there is no official documentation for the operations, other than the EXPLAIN documentation
14.2 Using EXPLAIN: https://www.postgresql.org/docs/current/using-explain.html
19.7 Query Planning: https://www.postgresql.org/docs/current/runtime-config-query.htm

[github] plannodes.h: https://github.com/postgres/postgres/blob/master/src/include/nodes/plannodes.h

[3rd party] Use the index Luke: https://use-the-index-luke.com/sql/explain-plan/postgresql/operations
[3rd party] depesz - explain part 3: https://www.depesz.com/2013/05/09/explaining-the-unexplainable-part-3/
[3rd party] depesz - explain part 4: https://www.depesz.com/2013/05/19/explaining-the-unexplainable-part-4/
[3rd party] EXPLAIN glossary: https://www.pgmustard.com/docs/explain

Gather
Gather merge
Parent node of **parallel** processes
Gather combines output from **Parallel** nodes
e.g. Gather combines result from **Parallel Seq** Scan

```
Gather  (cost=1000.00..647776.33 rows=20926 width=43)
(actual time=13.890..1490.239 rows=695974 loops=1)
   Output: ....
   Workers Planned: 2
   Workers Launched: 2
   ->  Parallel Append  (cost=0.00..644683.73 rows=8720
width=43) (actual time=3.656..1457.741 rows=231991
loops=3)
         Buffers: shared hit=557179
         Worker 0:  actual time=0.008..1531.405
rows=239114 loops=1
           Buffers: shared hit=190472
         Worker 1:  actual time=0.009..1497.295
rows=251270 loops=1
           Buffers: shared hit=187144
         ->  Parallel Seq Scan on ....
               Output: rep_rank_1.system_brand_code,
rep_rank_1.system_country_code, rep_rank_1.store_code,
rep_rank_1.sales_sum_code, rep_rank_1.week_start_date,
rep_rank_1.three_weeks_total_sales_result,
rep_rank_1.representative_sku_code
               Filter: ....
         ->  Parallel Seq Scan on ....
               Output: ....
               Filter: ....
         ->  Parallel Seq Scan on ....
               Output: ....
               Filter: ....
```

Gather Merge also respects (=preserves) **order**
e.g. Gather Merge to **Sort** in parallel

```
 Gather Merge  (cost=141863.64..152100.27 rows=57668
width=153) (actual time=221.248..296.356 rows=18886
loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=124320, temp read=1941
```

```
written=1948
    ->  GroupAggregate  (cost=140863.62..144443.92
rows=28834 width=153) (actual time=211.051..253.648
rows=6295 loops=3)
          Group Key: ...
          Buffers: shared hit=124320, temp read=1941
written=1948
          ->  Sort  (cost=140863.62..141163.98
rows=120145 width=43) (actual time=211.033..221.373
rows=96148 loops=3)
                Sort Key: ....
                Sort Method: external merge Disk: 5016kB
                Buffers: shared hit=124320, temp
read=1941 written=1948
                ->  Append  (cost=0.12..127030.73
rows=120145 width=43) (actual time=0.903..151.861
rows=96148 loops=3)
                      Buffers: shared hit=124304
                      ->  Seq Scan on ...
                            Filter: ...
                      ->  Seq Scan on ...
                            Filter: ...
                      ->  Seq Scan on ...
                            Filter: ...
```

Seq Scan
Sequential scan i.e. **loop** through each row

Index Scan
Index Only Scan
Index only scans: https://www.postgresql.org/docs/current/indexes-index-only-scans.html
Index used for **searching** instead of by Seq Scan.
Index Scan uses the index to determine the table row, and separately **retrieves the row** from the table.
Index Only Scan does NOT involve retrieving the row, by using **denormalized values** included in the index.

Bitmap Index Scan
Bitmap Heap Scan
Recheck Cond
https://use-the-index-luke.com/sql/explain-plan/postgresql/operations
Bitmap Index Scan is an optimization of **Index Scan** when **bulk accessing** the index according to some **uniform condition**.
Addresses sorted in a **bitmap**.
Similarly, **Bitmap Heap Scan** is an optimization of **Seq Scan** when accessing in some order.

e.g. optimize access for unique < 100 condition

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100;

                                  QUERY PLAN
-------------------------------------------------------------
```

```
----------------------
 Bitmap Heap Scan on tenk1  (cost=5.06..224.98 rows=100
width=244)
   Recheck Cond: (unique1 < 100)
   ->  Bitmap Index Scan on tenk1_unique1
(cost=0.00..5.04 rows=100 width=0)
         Index Cond: (unique1 < 100)
```

BitmapAnd
BitmapOr
Combining multiple indexes: https://www.postgresql.org/docs/current/indexes-bitmap-scans.html
Combined index

e.g. combine index of unique1 and unique2

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND
unique2 > 9000;

                                     QUERY PLAN
-----------------------------------------------------------
----------------------------
 Bitmap Heap Scan on tenk1  (cost=25.07..60.11 rows=10
width=244)
   Recheck Cond: ((unique1 < 100) AND (unique2 > 9000))
   ->  BitmapAnd  (cost=25.07..25.07 rows=10 width=0)
         ->  Bitmap Index Scan on tenk1_unique1
(cost=0.00..5.04 rows=100 width=0)
               Index Cond: (unique1 < 100)
         ->  Bitmap Index Scan on tenk1_unique2
(cost=0.00..19.78 rows=999 width=0)
               Index Cond: (unique2 > 9000)
```

NestedLoop (Join)
See JOIN strategies below.

Hash Join
HashJoin joins using the hash table created by Hash.
See JOIN strategies below.

e.g.

```
$ explain analyze select * from pg_class c join
pg_namespace n on c.relnamespace = n.oid;

QUERY PLAN
-----------------------------------------------------------
-----------------------------------------------------------
------
 Hash Join  (cost=1.14..16.07 rows=292 width=316)
(actual time=0.036..0.343 rows=295 loops=1)
```

```
   Hash Cond: (c.relnamespace = n.oid)
   ->  Seq Scan on pg_class c  (cost=0.00..10.92
rows=292 width=203) (actual time=0.007..0.044 rows=295
loops=1)
   ->  Hash  (cost=1.06..1.06 rows=6 width=117) (actual
time=0.012..0.012 rows=6 loops=1)
         Buckets: 1024  Batches: 1  Memory Usage: 1kB
         ->  Seq Scan on pg_namespace n
(cost=0.00..1.06 rows=6 width=117) (actual
time=0.004..0.005 rows=6 loops=1)
```

Merge Join
See JOIN strategies below.

Sort
[PostgresPro] Merge JOIN: https://postgrespro.com/blog/pgsql/5969770
e.g.
  ORDER BY
  JOIN with Merge Join

Sort types:

Quick sort	quick sort
External sort	- Quick sort: sorted all in memory (**work_mem**) - External sort: not enough memory for the entire sort. Use **temp files**.
	Starts as Quick sort node, and **switches** to External sort if memory becomes full.
Top-N heap sort	sort a **LIMIT N** set Just iterate through rows and keep the top N values.
Incremental sort	sort by **multiple** keys, when already sorted by a **subset** e.g. sort by (k1, k2, k3) when sorted by (k1, k2) already

Hash
Materialize
Memoize
[PostgresPro] Nested Loops: https://postgrespro.com/blog/pgsql/5969618
[PostgresPro] Hash JOIN: https://postgrespro.com/blog/pgsql/5969673
All are **row cache** in **memory**.

	Data structure	**Access**	**Max memory**
Materialize	table i.e. **list** of rows	seq scan	work_mem **spill to disk (temp file)** when full
Hash	hash table	key lookup	work_mem * hash_mem_mulitplier use **temp**

file when full

i.e. **split** hash table into multiple **temp files**

Memoize hash table key lookup work_mem * hash_mem_multiplier

replace **cold keys** when full

CTE Scan
Similar to Sequential Scan
Sequential scan on a materialized CTE.

Append
Combine results from **partitioned** tables.

e.g. partitions of partitioned table t_facing_span_settings

```
Append  (cost=0.00..3219.68 rows=548 width=73) (actual
time=27.765..33.277 rows=3438 loops=1)
   ->  Seq Scan on t_facing_span_setting_202003 face_1
    ...
   ->  Seq Scan on t_facing_span_setting_202004 face_2
    ...
   ->  Seq Scan on t_facing_span_setting_202005 face_3
    ...
   ->  Seq Scan on t_facing_span_setting_202006 face_4
    ...
```

Hash Aggregate
GROUP BY aggregation using a hash table.
See GROUP BY below.

Group Aggregate
Finalize Group Aggregate
GROUP BY aggregation using sort.
See GROUP BY below.

*** GROUP BY plan**
Indexing GroupBy: https://use-the-index-luke.com/sql/sorting-grouping/indexed-group-by
airbyte: https://airbyte.com/blog/postgresql-query-plans-for-aggregating-data

HashAggregate vs **GroupAggregate**
Similar to Hash vs Merge JOIN:
HashAggregate uses a **hash table** of the group key for aggregation.
GroupAggregate uses **sorting** by the group key for aggregation.

HashAggregate is often **faster** (sorting is expensive), but may consume a lot of **memory** and may lead to **swapping**.

Steps
a) **HashAggregate**
Group using a **hash table**, with the group columns as **hash keys**.

step 1	**build** hash table	1 sequential scan of source table
step 2	**aggregate** hash table	1 sequential scan of hash table

b) **GroupAggregate (+ Sort)**:
Group by **sorting** the table with the group columns as **sort keys**.

step 1	row source table with group keys	**n sequential scan** of source table
step 2	collapse sorted table	1 sequential scan of sorted table

Efficiency

	Efficiency	**Memory**
Hash	only 2 sequential scans	O(g) where g = number of groups i.e. efficient when few groups
Group	n >= 2 sequential scans	O(1)

e.g.
query

```
SELECT x, y, sum(n) FROM t GROUP BY x,y
```

HashAggregate

```
 Hash table

  x   y           n                                  x
y    sum(n)                       x   y    sum(n)
 [1]['a']    =   [1,2,3]      ==Aggregate==>        [1]
['a'] = 6            ==result==>      1   'a'   6
 [1]['b']    =   [11,12]                            [1]
['b'] = 23                        1   'b'   23
 [2]['a']    =   [...]                              [a]
['a'] = ...                       2   'a'   ...
 ...                                                ...
                                  ...
```

Sort + GroupAggregate

```
   SORT BY x, y

   x     y        n     == Aggregate ==>          x     y
sum(n)
   1     'a'      1                               1    'a'
6
   1     'a'      2                               1    'b'
23
```

```
   1    'a'     3
2   'a'   ...
   1    'b'     11                          ...
   1    'b'     12
   2    'a'     -1
   ...
```

```
> EXPLAIN ANALYZE SELECT passenger_name, count(*) AS n
FROM tickets GROUP BY passenger_name ;

Finalize GroupAggregate  (cost=44630.76..47219.48
rows=10218 width=24) (actual time=270.025..309.145
rows=27909 loops=1)"
  Group Key: passenger_name"
  ->  Gather Merge  (cost=44630.76..47015.12 rows=20436
width=24) (actual time=270.014..293.964 rows=61056
loops=1)"
        Workers Planned: 2"
        Workers Launched: 2"
        ->  Sort  (cost=43630.73..43656.28 rows=10218
width=24) (actual time=264.612..270.608 rows=20352
loops=3)"
              Sort Key: passenger_name"
              Sort Method: external merge  Disk: 728kB"
              Worker 0:  Sort Method: external merge
Disk: 720kB"
              Worker 1:  Sort Method: external merge
Disk: 776kB"
              ->  Partial HashAggregate
(cost=39474.60..42950.27 rows=10218 width=24) (actual
time=198.285..223.757 rows=20352 loops=3)"
                    Group Key: passenger_name"
                    Batches: 5  Memory Usage: 1073kB
Disk Usage: 2312kB"
                    Worker 0:  Batches: 5  Memory Usage:
1073kB  Disk Usage: 1760kB"
                    Worker 1:  Batches: 5  Memory Usage:
1073kB  Disk Usage: 3400kB"
                    ->  Parallel Seq Scan on tickets
(cost=0.00..17344.46 rows=345446 width=16) (actual
time=0.053..52.217 rows=276357 loops=3)"
Planning Time: 0.081 ms"
Execution Time: 311.872 ms"
```

==========
======

== JOIN strategies ==

50.5 Planner/optimizer: https://www.postgresql.org/docs/current/planner-optimizer.html

[Use the index Luke] operations: https://use-the-index-luke.com/sql/explain-plan/postgresql/operations
[Cybertech] JOIN strategies and performance: https://www.cybertec-postgresql.com/en/join-strategies-and-performance-in-postgresql/
[PostgresPro] Queries in PostgreSQL 1: Query execution stages: https://postgrespro.com/blog/pgsql/5969262

Driving (aka outer) table
which table is considered driving table: https://stackoverflow.com/questions/12177672/which-table-is-considered-inner-in-a-nested-loop-join
In a **JOIN**, at least one of the tables needs to be **looped** through (in general. In some cases, the outer may also be e.g. an Index Scan).
The table of the outermost is the **driving (outer) table**.
The choice of driving table does **NOT** depend on the JOIN order.
i.e. FROM a INNER JOIN b does NOT mean a is the driving table.

3 types of JOIN strategies

1) **Nested Loop JOIN**: Simplest JOIN method.
[PostgresPro] Queries in PostgreSQL 5: Nested loop: https://postgrespro.com/blog/pgsql/5969618
Nested because it **queries** the **inner table** at each step.
i.e. a **nested query** in a loop
In particular, it may repeat the **same query multiple times** (<=> Hash JOIN performs a query **only once**)

In pseudo code

```
for l in left {
  rs = SELECT ... FROM right WHERE key(l) = key(r)
  ...
}
```

e.g. Simple INNER JOIN

```
SELECT ...
FROM a                          -- driving table
INNER JOIN b ON a.id = b.id
```

Query plan

```
Nested Loop
  -> Seq Scan on a                -- driving table
  -> Index Scan on b
     Filter: a.id = b.id
```

e.g. INNER JOIN with a **filter**
Outer loop need not be a Seq Scan

```
SELECT ...
FROM a                            -- driving table
INNER JOIN b ON a.id = b.id
```

```
WHERE a.id = 10
```

Query plan

```
Nested Loop
  -> Index Scan on a                    -- driving table
     Filter: a.id = 10
  -> Index Scan on b
     Filter: a.id = b.id
```

e.g. multiple INNER JOINs

```
SELECT ...
FROM a
INNER JOIN b ON a.id = b.id
INNER JOIN c ON b.id = c.id
```

Query plan

```
Nested loop
  -> Seq Scan on a
  -> Nested loop
    -> Index Scan on b                  -- outer table is
also Index Scan
       Filter: b.id = a.id
    -> Index Scan on c
       Filter: b.id = c.id
```

2) **HASH JOIN**: a cached version of nested loop using a **hash table**.
i.e. Loops through the driving table and checks the **inner hash table** at each step.
Unlike, nested JOIN, does NOT need to repeat the same query.
Requires **memory** (**temp file** if not enough) to build the hash table.

In pseudo code

```
hashB = {}
for b in right {
  hashB[key(b)] = (hashB[key(b)] || []) + [b]
}

for a in ta {
  bs = hashB[key(a)]
  ...
}
```

e.g. INNER JOIN on a column with **NO index**

```
SELECT ...
FROM a                              -- driving table
INNER JOIN b ON a.name = b.name     -- no index
```

Query plan

```
Hash Join
  Hash Cond: a.name = b.name
  -> Seq Scan on a                  -- driving table
  -> Hash
     -> Seq Scan on b
```

If the columns are indexed, then a Nested Loop is more efficient

```
SELECT ...
FROM a                              -- driving table
INNER JOIN b ON a.id = b.id         -- has index
WHERE a.id = 10
```

Query plan

```
Nested Loop
  -> Seq Scan on a                  -- driving table
  -> Index Scan on b
     Filter: a.id = b.id
```

- **Merge JOIN**: JOIN similar to **zipping**
i.e. step through both tables **at the same time**, and join if they match.
Requires both tables to be **sorted**.
Efficient since **O(n)** n = max of sides

e.g.

```
while a && b {
  if a.id == b.id {
    ... join ...
    a = a.next()
    b = b.next()
  } else if a.id < b.id {
    a = a.next()
  } else if a.id > b.id {
    b = b.next()
  }
}
```

e.g. JOIN with **ORDER BY**
If the final result needs to be ordered anyway,
may be more efficient to order **first**, then JOIN.

```
SELECT ...
```

```
FROM a                                -- driving table
INNER JOIN b ON a.id = b.id
WHERE a.id = 10
ORDER BY a.id
```

Query plan

```
Merge Join
  Merge Cond: a.id = b.id
  -> Sort
     Sort Key: a.id
       -> Seq Scan on a
     Sort Key: b.id
       -> Seq Scan on b
```

Nested Loop JOIN vs Hash JOIN vs Merge JOIN
[Cybertec] join strategies: https://www.cybertec-postgresql.com/en/join-strategies-and-performance-in-postgresql/#inner-and-outer-relation
[PostgresPro] Sort and merge - comparison of join methods: https://postgrespro.com/blog/pgsql/5969770
[Cybrosis] How Postgres chooses JOIN performances: https://www.cybrosys.com/research-and-development/postgres/how-postgresql-chooses-join-algorithms
[zenn.dev] Nested loop/hash/sort merge結合の違いとパフォーマンス比:
https://zenn.dev/loglass/articles/84f15be9a4d2c9
[SO] Nested loop vs merge join vs hash join in Postgres: https://stackoverflow.com/questions/49023821/nested-join-vs-merge-join-vs-hash-join-in-postgresql

	Requirement	**Description**
Nested Loop	-	most common and default method used BAD: **inner table** is a Seq Scan (or equivalent)
Hash JOIN	**Hash** of inner table cost: **work_mem** * **hash_multiplier**	**No Index** required GOOD: **small hash table** that fits in memory temp file if spill BAD: not enough **memory** for hash table
Merge JOIN	both tables sorted either **Index Scan** (sorted) or **Sort** node cost: **work_mem** for sorting if needed	**No Index** required GOOD: **stable** performance since no **driving table** i.e. choice of inner/outer table does not matter.

Optimal if both tables are **sorted**, or sorting is cheap.

BAD:

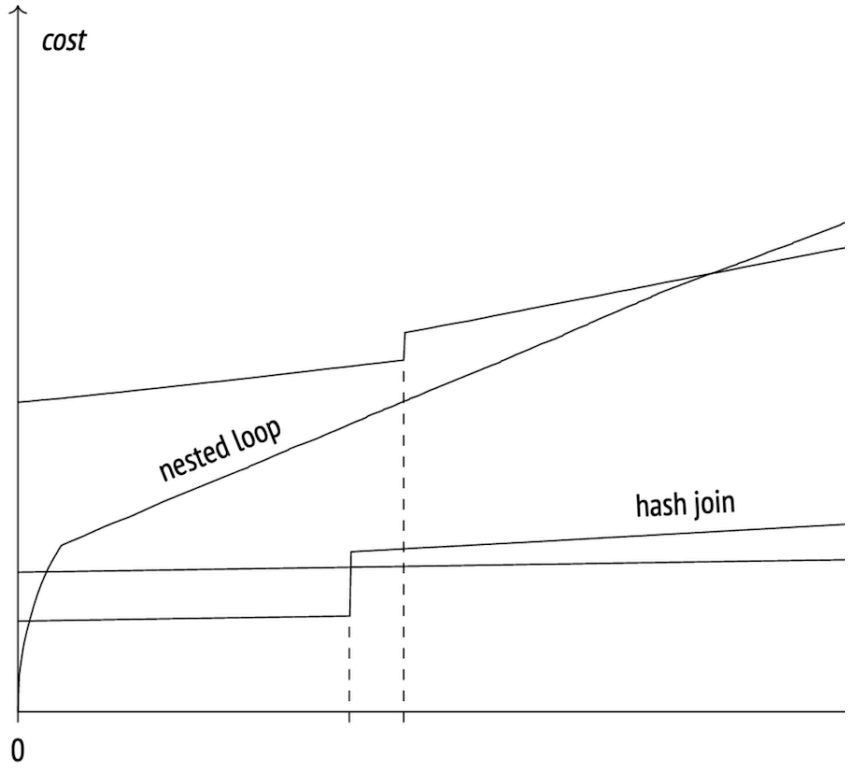
not enough **memory** for sorting in memory

Performance
e.g.

		Complexity **Performance** n = outer table size m = inner table size s = selectivity	**Case**
Nested Loop		O(nm) **BAD**	outer: Seq Scan inner: Seq Scan
		O(n log m)	outer: Seq Scan inner: Index Scan
	OK		
		O(s (log n + m)) = O(s log n + sm)	outer: Index Scan inner: Seq Scan
	BAD		
		O(s (log n + log m)) = O(s log n + s log m)	outer: Index Scan inner: Index Scan
	OK		
Hash Join		O(n + m) OK m = building hash table **BAD** if hash table size >> work_mem * hash_mulitiplier	
Merge Join		O(n + m) OK	both table sorted
		O(n log n + m) OK	one table sorted
		BAD if Sort memory >> work_mem	
		O(n log n + m log m) OK	neither table sorted
		BAD if Sort memory >> work_mem	

Selectivity (0 ~ 1) = **relative amount** of rows in whole table queried
e.g. selectivity 0.1 = 10% of whole table

The image below illustrates approximate costs of different join methods at vari



*** NOTE: JOIN order in query plan (join_collapse_limit, from_collapse_limit)**
[CYBERTEC] Forcing a join order: https://www.cybertec-postgresql.com/en/forcing-a-join-order-in-postgresql/
[pganalyze] Forcing a join order: https://pganalyze.com/blog/5mins-postgres-forcing-join-order

NOTE: Only **2 JOINs** are processed at a time
https://use-the-index-luke.com/sql/join

There is, however, one thing that is common to all join algorithms: t
at a time. A SQL query with more tables requires multiple steps: firs
result set by joining two tables, then joining the result with the next

Pipelining Intermediate Results

Although intermediate results explain the algorithm very well, it
database has to materialize it. That would mean storing the interr
join before starting the next one. Instead, databases use pipelinin
That means that each row from the intermediate result is immedi
join operation—avoiding the need to store the intermediate result

JOIN order
JOIN collapse

FROM collapse
Explicit JOINs: https://www.postgresql.org/docs/current/explicit-joins.html
join_collapse_limit, from_collapse_limit: https://www.postgresql.org/docs/current/runtime-config-query.html#GUC-JOIN-COLLAPSE-LIMIT

In general, the **order of JOINs** in the SQL **syntax** is **insignificant**.
i.e. **query planner** considers **all possible JOIN orders** to search for the most optimal.

To be precise, this is controlled by the **collapse_limit** settings
Higher limit = higher number of rearrangements to consider
i.e. higher chance of finding **optimal** plan but more **expensive** plan time.

join_collapse_limit — Max size of a list of JOINs that can be rearranged
default: 8

1 = no collapse

e.g. 3 JOINs

```
FROM a
INNER JOIN b ON ...
INNER JOIN c ON ...
```

from_collapse_limit — When collapsing JOINs, subqueries may be **collapsed** to parent
default: 8 — Allow collapse if the resulting list of JOINs does not exceed this limit.

1=no collapse

e.g.

```
FROM a                    =>
FROM a
INNER JOIN (
  SELECT ...
  FROM b
  INNER JOIN c ON ...
)
INNER JOIN b ON ...
INNER JOIN c ON ...
```

e.g. **join_collapse_limit**

SQL: with JOIN and FROM collapse

```
set join_collapse_limit TO 3;

SELECT *
FROM a
INNER JOIN b ON a.id = b.id
INNER JOIN c ON b.id = c.id

SELECT *                          -- same plan by JOIN collapse
FROM b
INNER JOIN c ON c.id = b.id
INNER JOIN a ON a.id = c.id
```

```
SELECT *                            -- same plan by FROM
collapse
FROM a
INNER JOIN
  (SELECT b.id FROM b
   INNER JOIN c ON b.id = c.id) d
ON d.id = a.id
```

Query plan: all possible orders are considered
i.e. any one is possible

```
Nested Loop Join                    -- (a b) c
  -> Nested Loop
    -> Seq Scan on a
    -> Index Scan on b
       Filter: a.id = b.id
  -> Index Scan on c
     Filter: b.id = c.id

Nested Loop Join                    -- (b a) c
  -> Nested Loop
    -> Seq Scan on b
    -> Index Scan on a
       Filter: a.id = b.id
  -> Index Scan on c
     Filter: b.id = c.id


Nested Loop Join                    -- (b c) a
  -> Nested Loop
    -> Seq Scan on b
    -> Index Scan on c
       Filter: c.id = b.id
  -> Index Scan on a
     Filter: a.id = b.id


Nested Loop Join                    -- (c b) a
  -> Nested Loop
    -> Seq Scan on c
    -> Index Scan on b
       Filter: c.id = b.id
  -> Index Scan on a
     Filter: a.id = b.id

Nested Loop Join                    -- a (b c)
  -> Seq Scan on a
  -> Nested Loop
    -> Index Scan on b
       Filter: a.id = b.id
    -> Index Scan on c
       Filter: a.id = c.id
```

```
Nested Loop Join                  -- a (c b)
  -> Seq Scan on a
  -> Nested Loop
    -> Index Scan on c
       Filter: a.id = c.id
    -> Index Scan on b
       Filter: a.id = b.id

Nested Loop Join                  -- b (a c)
  -> Seq Scan on b
  -> Nested Loop
    -> Index Scan on a
       Filter: a.id = b.id
    -> Index Scan on c
       Filter: b.id = c.id

... etc
```

SQL: no collapse

```
set join_collapse_limit TO 1;

SELECT *
FROM a
INNER JOIN b ON a.id = b.id
INNER JOIN c ON b.id = c.id;
```

Possible query plans:

```
Nested Loop Join                  -- (a b) c
  -> Nested Loop
    -> Seq Scan on a
    -> Index Scan on b
       Filter: a.id = b.id
  -> Index Scan on c
     Filter: b.id = c.id

Nested Loop Join                  -- (b a) c
  -> Nested Loop
    -> Seq Scan on b
    -> Index Scan on a
       Filter: a.id = b.id
  -> Index Scan on c
     Filter: b.id = c.id


Nested Loop Join                  -- a (b c)
  -> Seq Scan on a
  -> Nested Loop
    -> Index Scan on b
       Filter: a.id = b.id
```

```
    -> Index Scan on c
       Filter: a.id = c.id


Nested Loop Join                  -- a (c b)
  -> Seq Scan on a
  -> Nested Loop
    -> Index Scan on c
       Filter: a.id = c.id
    -> Index Scan on b
       Filter: a.id = b.id
```

*** NOTE: JOIN order by JOIN type (INNER/OUTER)**
Explicit JOINs: https://www.postgresql.org/docs/current/explicit-joins.html

Although the **JOIN order** in the SQL syntax is insignificant (query planner will consider all possible rearrangements)
the **type** of JOIN may still limit the possible plans.

e.g. **LEFT OUTER JOIN**: LEFT table must be the driving table

INNER JOIN: any table may be driving table

```
SELECT *
FROM a
INNER JOIN b ON a.id = b.id
INNER JOIN c ON b.id = c.id
```

Possible query plans

```
Nested Loop Join
  -> Nested Loop
    -> Seq Scan on a          -- a is driving table
    -> Index Scan on b
       Filter: a.id = b.id
  -> Index Scan on c
     Filter: b.id = c.id

Nested Loop Left Join
  -> Nested Loop
    -> Seq Scan on b          -- b is driving table
    -> Index Scan on a
       Filter: a.id = b.id
  -> Index Scan on c
     Filter: b.id = c.id

...
```

LEFT OUTER JOIN

```
SELECT *
FROM a                    -- a is driving table
LEFT OUTER JOIN
   (
     SELECT b.id
     FROM b
     INNER JOIN c
     ON b.id = c.id       -- For each row of a, process
JOIN of b and c with b.id = c.id = a.id
   ) d                    -- Then join with row of a, if
it exists.
ON a.id = d.id
```

Query plan:

```
Nested Loop Left Join
  -> SeqScan on a             -- a is driving table
  -> Nested Loop
    -> Index Scan on b
       Filter: a.id = b.id
    -> Index Scan on c
       Filter: b.id = c.id

Nested Loop Left Join
  -> Nested Loop Left Join
     -> Seq Scan on a         -- a is driving table
     -> Index Scan on b
        Filter: a.id = b.id
  -> Index Scan on c
     Filter: b.id = c.id
```

==========
======

== EXPLAIN result==

14.2 Using EXPLAIN: https://www.postgresql.org/docs/current/using-explain.html

EXPLAIN result

Tree of **plan node** executions.

e.g.

```
> EXPLAIN (ANALYZE, BUFFERS, VERBOSE) SELECT * FROM
pg_tables;

QUERY PLAN
──────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────
```

```
_____________________________________________________
 Hash Left Join  (cost=2.14..1236.46 rows=4285
width=260) (actual time=0.017..3.562 rows=2435 loops=1)
   Output: n.nspname, c.relname,
pg_get_userbyid(c.relowner), t.spcname, c.relhasindex,
c.relhasrules, c.relhastriggers, c.relrowsecurity
   Inner Unique: true
   Hash Cond: (c.reltablespace = t.oid)
   Buffers: shared hit=846
   ->  Hash Left Join  (cost=1.09..1189.63 rows=4285
width=140) (actual time=0.014..3.227 rows=2435 loops=1)
         Output: c.relname, c.relowner, c.relhasindex,
c.relhasrules, c.relhastriggers, c.relrowsecurity,
c.reltablespace, n.nspname
         Inner Unique: true
         Hash Cond: (c.relnamespace = n.oid)
         Buffers: shared hit=845
         ->  Seq Scan on pg_catalog.pg_class c
(cost=0.00..1165.38 rows=4285 width=80) (actual
time=0.004..2.856 rows=2435 loops=1)
               Output: c.oid, c.relname, c.relnamespace,
c.reltype, c.reloftype, c.relowner, c.relam,
c.relfilenode, c.reltablespace, c.relpages, c.reltuples,
c.relallvisible, c.reltoastrelid, c.relhasindex,
c.relisshared, c.relpersistence, c.relkind, c.relnatts,
c.relchecks, c.relhasrules, c.relhastriggers,
c.relhassubclass, c.relrowsecurity,
c.relforcerowsecurity, c.relispopulated, c.relreplident,
c.relispartition, c.relrewrite, c.relfrozenxid,
c.relminmxid, c.relacl, c.reloptions, c.relpartbound
               Filter: (c.relkind = ANY
('{r,p}'::"char"[]))
               Rows Removed by Filter: 6102
               Buffers: shared hit=844
         ->  Hash  (cost=1.04..1.04 rows=4 width=68)
(actual time=0.005..0.005 rows=9 loops=1)
               Output: n.nspname, n.oid
               Buckets: 1024  Batches: 1  Memory Usage:
9kB
               Buffers: shared hit=1
               ->  Seq Scan on pg_catalog.pg_namespace n
(cost=0.00..1.04 rows=4 width=68) (actual
time=0.001..0.002 rows=9 loops=1)
                     Output: n.nspname, n.oid
                     Buffers: shared hit=1
   ->  Hash  (cost=1.02..1.02 rows=2 width=68) (actual
time=0.001..0.002 rows=2 loops=1)
         Output: t.spcname, t.oid
         Buckets: 1024  Batches: 1  Memory Usage: 9kB
         Buffers: shared hit=1
         ->  Seq Scan on pg_catalog.pg_tablespace t
(cost=0.00..1.02 rows=2 width=68) (actual
time=0.001..0.001 rows=2 loops=1)
               Output: t.spcname, t.oid
```

```
               Buffers: shared hit=1
 Planning:
   Buffers: shared hit=514
 Planning Time: 0.510 ms
 Execution Time: 3.707 ms
(33 rows)
```

actual time=s...t loops=l

s **startup time (milliseconds)**: time until the **first row** is emitted from the **plan node**

Relative to **start** of current **node execution**

i.e. time 0 = start of

startup time is **low** if child nodes are **incremental** i.e. can emit **partial results**.

e.g.

Seq Scan	low	can emit row immediately as they are read
Sort	high	no output until all the rows have been sorted

t **total time (milliseconds)**: time of plan node execution end

i.e. **last row** is emitted

l number of times the plan node is executed.

If l > 1, then start and total time are **average times** per execution.

e.g. actual time of **Sort**

time	**process**
0	Sort Start -> Nested Loop JOIN Start ` -> Seq Scan start
0.017 ms	Seq Scan emit first row
0.027 ms	Seq Scan emit last row -> Seq Scan done
0.077 ms	Nested Loop JOIN emit first row
2.396 ms	Nested Loop JOIN emit last row -> Nested Loop JOIN done -> Sort begin sorting
3.069 ms	Sort end sorting -> Sort emit first row
3.084 ms	Sort emit last row -> Sort done

```
 ->  Sort  (cost=224.08..224.08 rows=1 width=190)
(actual time=3.069..3.084 rows=35 loops=1)
   Output: ...
   Sort Key: bc.system_brand_code,
bc.system_country_code, obj.floor_id, obj.object_id
```

```
   Sort Method: quicksort  Memory: 142kB
   Buffers: shared hit=916
   ->  Nested Loop  (cost=0.41..224.07 rows=1 width=190)
(actual time=0.077..2.396 rows=832 loops=1)
         Output: ...
         Buffers: shared hit=916
         ->  Seq Scan on sch_slp.m_brand_country bc
(cost=0.00..11.38 rows=1 width=114) (actual
time=0.017..0.027 rows=11 loops=1)
               Output: ...
               Filter: ((bc.system_business_code)::text
= '7'::text)
               Rows Removed by Filter: 21
               Buffers: shared hit=1
         ->  Index Scan using
t_blank_map_layout_999912_pkey on
sch_slp.t_blank_map_layout_999912 obj
(cost=0.41..211.87 rows=82 width=76) (actual
time=0.008..0.203 rows=76 loops=11)
               Output: ....
               Index Cond: ....
               Buffers: shared hit=915
```

e.g. actual time of **loop > 1**

0 ms	Top Nested Loop JOIN A start -> Outer Nested Loop JOIN B start
7.427 ms	Outer Nested Loop JOIN B emit first row
9.149 ms	Outer Nested Loop JOIN B emit last row -> Outer Nested Loop JOIN B done
22 * 2138.767 ms ~ 52179 ms	Inner Nested Loop JOIN C all 22 rows done -> Inner Nested Loop JOIN C done
52189.224 ms	Top Nested Loop JOIN emit last row -> Top Nested Loop JOIN

```
->  Nested Loop Left Join  (cost=96138.43..114038.68
rows=1 width=174) (actual time=2299.129..52189.225
rows=22 loops=1)     Nested Loop A
       Output: ....
       Buffers: shared hit=496721, temp read=35099
written=35763
       ->  Nested Loop  (cost=1386.60..1397.22 rows=1
width=163) (actual time=7.427..9.149 rows=22 loops=1)
                     Nested Loop B
             Output: ....
             Inner Unique: true

        ....
```

```
       ->  Nested Loop Left Join
(cost=94751.83..112641.41 rows=4 width=25) (actual
time=2138.767..2371.817 rows=1 loops=22)   Nested Loop C
             Output: ....
             Buffers: shared hit=494917, temp read=35099
written=35763
             ->  Nested Loop  (cost=94751.41..112639.94
rows=1 width=28) (actual time=2138.743..2371.785 rows=1
loops=22)
                   Output: ....
                   Buffers: shared hit=494720, temp
read=35099 written=35763
```

cost=s...t
s (relative) estimate of actual startup time
t (relative) estimate of actual total seconds

rows=r
Number of rows returned by the operation

width=w
Number of **bytes** per row

shared hit=h read=r dirtied=d written =w
[BoringSQL] Reading buffer statistics: https://boringsql.com/posts/explain-buffers/
Shared buffer usage in # of **pages**.

hit = h	h pages read from shared buffer i.e. **cache hit**
read = r	r pages read from file into shared buffer i.e. **cache miss**
dirtied = d	Write operations are applied on **cache** first = **dirtied** pages
written = w	Then written to file in background = **write** pages

local hit=h read=r written=w
Same as shared, but for **TEMPORARY** tables
i.e. # of pages used by temp tables.

temp read=r written =w
Swap usage when not enough **work_mem**.

written =w	w pages written to swap
read=r	r pages read from swap

e.g. Buffers in **Planning**
Buffers used by the query planner when accessing the **system catalog**
e.g. access **pg_class**, **pg_stats**, etc

```
 Nested Loop Left Join  (cost=1057.99..3422.16 rows=14
width=401) (actual time=4.758..11.195 rows=607 loops=1)
   Output: ....
```

```
Planning:
  Buffers: shared hit=259 read=2
Planning Time: 2.187 ms
Execution Time: 11.476 ms
```

*** Example: actual cost**

```
============================================================
                            ======
```

```
This
is
code
Monaco 14
```